Practice Session IV: Macros, Python Models & Testing

Analytics Engineering · Session 16 · ~1 hour

Goal

Consolidate yesterday's session: write macros from scratch, build a Python model, and add a full test suite to your work.

By the end you will have:

- A new macro using Jinja `if/elif/else` and default arguments
- A Python model (Pandas) that analyzes website traffic
- A full test suite — generic, singular, and custom generic tests
- A clean `dbt build` with zero failures

Checklist

- ☐ `macros/classify_revenue.sql` — macro with `if/elif/else` + default arguments
- ☐ `models/marts/mart_revenue_tiers.sql` — SQL model that uses the macro twice
- ☐ `models/marts/mart_channel_performance.py` — Pandas Python model
- ☐ `models/marts/_new_models.yml` — tests for both new models
- ☐ `tests/assert_channel_conversion_rate_is_valid.sql` — singular test
- ☐ `dbt build -s mart_revenue_tiers mart_channel_performance` passes with zero failures

Exercise 1 — Macro (20 min)

Part A: Write the macro

Create `macros/classify_revenue.sql`.

The macro takes an `amount_col` argument and returns a `case` expression that assigns `'low'`, `'medium'`, or `'high'` based on thresholds.

- Default thresholds: `low = 100`, `high = 500`
- Callers must be able to override the thresholds at the call site

Part B: Use the macro in a model

Create `models/marts/mart_revenue_tiers.sql` using `int_orders_enriched` as the source.

Include only completed orders. Pass through `order_id`, `customer_id`, `order_date`, `status`, `total_amount`, and add:

Column	Description
`revenue_tier`	Classified with **default** thresholds
`revenue_tier_strict`	Classified with `low=50, high=200`

Exercise 2 — Python Model (25 min)

Create `models/marts/mart_channel_performance.py` using `stg_website_sessions`.

Available columns: `session_id`, `source`, `device_type`, `page_views`, `session_duration_seconds`, `is_bounce`, `converted`.

Group by `source` and `device_type`. The model must produce:

Column	Description
`source`	Traffic source
`device_type`	
`total_sessions`	Count of sessions
`conversions`	Count of converted sessions
`avg_page_views`	
`avg_duration_seconds`	
`bounce_rate`	Share of bounced sessions
`conversion_rate`	`conversions / total_sessions * 100`
`channel_quality`	`'high_performing'` ≥ 10%, `'average'` ≥ 5%, else `'low_performing'`

Exercise 3 — Tests (15 min)

Generic tests

Add `models/marts/_new_models.yml` with tests for both models:

- **mart_revenue_tiers**: `order_id` unique and not null; `revenue_tier` and `revenue_tier_strict` not null and only `'low'`, `'medium'`, `'high'`; `total_amount` positive
- **mart_channel_performance**: `source` and `device_type` not null and only known values; `conversion_rate` and `channel_quality` not null; `conversion_rate` positive

Singular test

Create `tests/assert_channel_conversion_rate_is_valid.sql`:

- Business rule: `conversion_rate` must be between 0 and 100
- The query must return the rows that violate the rule (if any)

Verify

Run the full build:

```
dbt build -s mart_revenue_tiers mart_channel_performance
```

All models and tests must pass. To inspect the SQL your macro generated before running:

```
dbt compile -s mart_revenue_tiers
```

Bonus — Extend the Macro

Rewrite `classify_revenue` so that thresholds and labels are passed as **lists** instead of individual arguments. The caller should be able to pass any number of thresholds and matching labels without changing the macro itself. Use a Jinja `for` loop to generate the `case` branches dynamically.